Train Scheduling with ASP: Acceleration and Deceleration

Jonas Nicolaysen and Roman Kalenov
University of Potsdam, Germany

Summer Semester 2025

Abstract

Despite its long history, train scheduling remains challenging, made even harder by growing route density and train numbers. In the University of Potsdam's Railway Scheduling project [1], we address the basic task of scheduling trains to prevent collisions and ensure timely arrivals, and extend it into the previously unaddressed areas by modeling acceleration and deceleration to make simulations more realistic and reduce costly complete stops.

Keywords: Flatland; Scheduling; Answer-Set Programming.

1 Introduction

This report presents our semester-long work on train scheduling and its extension using Answer Set Programming, outlining key milestones without unnecessary detail. It includes code snippets and graphics that help intuitively understand the abstractions in our solutions.

2 Motivation

Our motivation stemmed from a shared interest in practical problem-solving and the counterintuitive idea of using answer sets to model motion, typically addressed with reinforcement learning or mathematical models.

3 Previous Work on the Problem

Railway scheduling has engaged scientists and engineers since the advent of trains, evolving from handwritten timetables and telegraph-based scheduling to mathematical tools like Gantt charts and modern AI-assisted automation. Answer Set Programming fits into this latter category due to its declarative nature and ability to model complex constraints transparently—unlike machine learning, whose inherent stochasticity is unacceptable in safety-critical contexts such as railway planning.

Foundational ASP-based work on train scheduling includes Train Scheduling with Hybrid ASP [2]. Of particular interest was prior student work on both domain extensions we pursued: prioritization [3] and speed [4].

4 Basic Problem

The core task in the railway scheduling project is to run an arbitrary number of trains without collisions while ensuring they arrive no later than their scheduled times. This challenge alone demands significant time and effort.

We began with two separate encodings, which helped us understand the domain better and compare two distinct solutions to identify the more effective one.

One solution minimized grounding with tight constraints and explicit movement rules, keeping the encoding close to the problem. The other adopted a more experimental approach with greater abstraction and modularity. For convenience, we call the first **'cell-based'** and the second **'trajectory-based'**.

The plan was to select the best encoding after a few weeks, using solving speed as the primary criterion. We now detail each approach to the basic problem.

4.1 Input/Output Format

The project allows teams complete freedom, with only the input and output format as constraints. The input is provided as a set of facts:

```
cell((Y, X), TRACK_TYPE).  % Y - vertical coordinate, top to bottom
                           % X - horizontal coordinate, left to right

train(TRAIN_ID).

start(TRAIN_ID, (Y, X), START_TIME, START_DIR).

end(TRAIN_ID, (Y, X), START_TIME, START_DIR).
```

Listing 1: Input facts for the ASP train scheduling instance

Two unusual properties of the input grid should be noted, **especially for future researchers starting their own experiments**:

- The first coordinate in the cell tuple represents vertical offset, not horizontal as in most coordinate systems.
- Coordinates start at the upper-left corner (inverted system), so the grid appears as in Figure 1.

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)

Figure 1: An example of input environment coordinates

This caused confusion in both encodings, forcing us to spend considerable time adapting our models to handle the unusual axis orientation in the provided test environments[1].

The model output must be a set of 'action(TRAIN, MOMENT, ACTION)' atoms, where ACTION is one of 'move forward', 'turn right', 'turn left', 'wait' and MOMENT corresponds to a time step within the global horizon given by the 'global' fact.

5 Cell-Based Solution

The cell-based encoding aimed to stay close to the problem description, modeling train movement explicitly in space and time. This low-level approach emphasizes rule precision and direct constraints over abstraction.

The encoding uses predicates like at/5 and move/5 to represent each train's position, direction, and movement at every time step. Movements are explicitly checked against the track type, and rules reflect the physical constraints of the railway network in the Flatland environment.

One main challenge was ensuring trains moved according to switches and crossings. Much of the encoding consists of track-specific rules that determine valid directions based on the cell's track type. This made the encoding lengthy but highly transparent: every train move is traceable through rules that specify how and under what conditions it occurs, easing debugging.

A unique feature of this encoding is its approach to waiting. Trains may wait only if an adjacent cell in their facing direction is occupied by a higher-priority train. This is enforced via a cardinality constraint that counts nearby trains and disallows waiting otherwise:

```
:- action(train(ID),wait,T), at(ID,X,Y,T,Dir),
    #count { ID2,DX,DY : offset(Dir,DX,DY), at(ID2,X+DX,Y+DY,T,_),
    ID>ID2 } = 0.
```

This logic prevents unnecessary waiting and train blockages. Movement rules reflect each track type; for example, a horizontal track of type 1025 allows only east–west movement:

```
move(ID,X,Y+1,T+1,e) :- action(train(ID), move_forwards,T),
    at(ID,X,Y,T,e), not goal(ID,T),
    cell((X,Y),1025), cell((X,Y+1),_), time(T), dir(e).
```

Similar rules apply to vertical tracks, switches, and crossings. The approach scales by adding cases rather than generalizing, keeping logic simple and debugging easy, but increasing length and rigidity. To prevent collisions, the encoding forbids two trains from occupying the same cell simultaneously or swapping positions in a single timestep:

```
:- at(ID1,X,Y,T,_), at(ID2,X,Y,T,_), ID1!=ID2.
:- at(ID1,X1,Y1,T,_), at(ID2,X2,Y2,T,_),
   at(ID1,X2,Y2,T+1,_), at(ID2,X1,Y1,T+1,_), ID1!=ID2.
```

Finally, the entire solution space is minimized based on the number of actions, prioritizing plans that reach the destination as efficiently as possible without unnecessary movement:

```
#minimize {1,ID,A,T : action(train(ID),A,T)}.
```

This cell-based encoding was surprisingly fast, outperforming more abstract approaches in raw solving time. Its efficiency likely stemmed from grounding only what was needed with very concrete rules for each movement. However, this came at the cost of flexibility: handling new track types or smarter waiting behavior required disproportionate adaptation. Still, its rule-by-rule structure ensured clear traceability and straightforward debugging.

6 Trajectory-Based Solution

Having worked on four prior ASP projects[5], we already had intuitions for structuring an ASP program to minimize grounding and increase modularity. The core idea was to reduce the search space before using the choice rule, which also lessened the number of constraints needed.

For readers interested in our full thought process and intermediate decisions, a weekly project journal is available in our repository.

6.1 Early Prototyping

To quickly grasp the domain problem, we implemented a simple Python rule-based pathfinder:

```
while current_segment != goal:
    track_type = railroad[current_segment]
    if track_type == 4 and len(action_log)==0:
        current_segment += 1
        action_log.append('east')
        timer += 1
        dispatcher.append((train, timer, current_segment))
        print(f"Train {train} on {current_segment} at moment {timer}")
    elif ...
```

Besides introducing us to the problem in familiar procedural programming terms, its value lies in highlighting the efficiency of the answer set approach to scheduling problems: the early ASP prototype capable of handling the same simple environment was **three times** shorter (60 lines vs. 160 lines of Python) and several times faster to implement:

```
max_time(MAX_TIME) :- MAX_TIME = #count{VAL : segment(VAL, TYPE), TYPE != 0}.

moment(1..MAX_TIME) :- max_time(MAX_TIME).

{ travel(MOMENT, POSITION) : moment(MOMENT),
segment(POSITION, TYPE), TYPE != 0 }
= MAX_TIME :- max_time(MAX_TIME).

:- travel(MOMENT, POSITION_1),
   travel(MOMENT, POSITION_2),
   POSITION_1 != POSITION_2.
```

6.2 Valid Trajectory Mining

The high-level intuition behind the search-space reduction mentioned earlier can be phrased as follows: rather than taking all input cells and feeding them into the generator, identify which cells can form a valid moving path in the input environment and generate rules only over these potential paths. In formal computer science terms, this can be described as:

Let X be the environment of size $m \times n$, with t denoting the number of trains present in the environment. Let T be the global time horizon, which no train is allowed to exceed.

In the worst-case scenario—where one train reaches its destination at the last moment allocated within the time horizon T—the computational complexity is given by:

$$\mathcal{O}_{\text{cells}} = m \times n \times t \times T.$$

Now let p be the number of valid trajectories mined from X. Although p can be large, it is always bounded above by the total number of environment cells:

$$p < m \times n.$$

The complexity using mined trajectories becomes:

$$\mathcal{O}_{\text{trajectories}} = p \times t \times T.$$

Since $p < m \times n$, it follows directly that:

$$\mathcal{O}_{\text{trajectories}} < \mathcal{O}_{\text{cells}}.$$

Thus, pre-mining trajectories strictly reduces computational complexity compared to exhaustive search, assuming t and T are fixed. With this established, we now turn to the technical aspects of the trajectory-based approach.

6.3 Flattening the Environment

A small module was written to convert the input 2d environments into 1d, with two objectives in mind:

1. Reducing cognitive load on the developer by twice reducing the delta calculation.
2. Slightly reducing grounding: instead of two variables, only one per valid trajectory cell is grounded (O-calculation omitted for brevity).

The conversion was implemented using a simple empirically derived formula:

$$\texttt{Flat Index} = \texttt{Column Index} \times \texttt{Grid Width} + \texttt{Row Index}$$

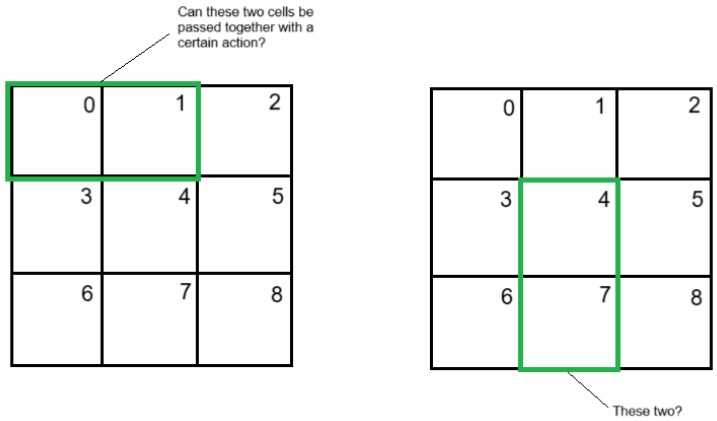
Which in ASP looked as following:

```
grid_size(ROWS, COLUMNS) :-
ROWS = #count{VAL : cell((VAL, _), _)},
COLUMNS = #count{VAL : cell((_, VAL), _)}.
cell_flat(Y * WIDTH + X, TYPE) :-
cell((Y, X), TYPE), grid_size(_, WIDTH).
```

6.4 'Path' Atoms

Instead of directly using the raw 2d input or its converted 1d version, we first created a layer of passable two cell 'paths' *clenched between each other* via a unique combination of the outgoing and incoming positions/directions.

This is somewhat analogous to applying convolutions to a 2D input in machine learning, where information is absorbed and condensed through a sliding window. The image below illustrates this process:



In terms of asp, this operation is a normal rule (see Table 1).

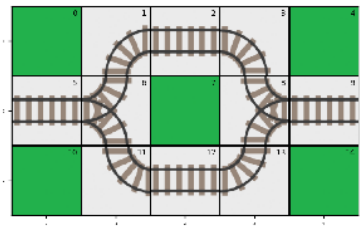
Table 1: A normal rule creating 'path' atoms from jointly passable input cells

Normal rule idea	**Example ASP Implementation**
If there are two cells in the input environment that can be passed by a single train using a certain action, then these cells must be united into a 'path' atom containing their indexes, the (unique) action that joins them and the directions that the train will follow as a result.	`path(north, POSITION, move_forward, north, POSITION - WIDTH) :- cell_flat(POSITION, 32800), grid_size(_, WIDTH), cell_flat(POSITION - WIDTH, TYPE), TYPE != 0.`

In addition to condensing the input space, this operation assigns a single action to each pair of cells absorbed into one atom. Unlike 'blind' generation over all possible actions, this hard mapping further reduces the search space: if we reduce the number of possible actions at each timestep from 4 to 1 (limiting waiting to junctions only), the total branching factor per train per timestep becomes **constant**.

The mined 'path' atoms together form **'trajectories'**—unique ordered sets of path atoms passable by a single train. A simple example illustrates this process.

Given a simple input environment:



```
cell((2,0,), 0).
cell((2,1), 72).
cell((2,2), 1025).
cell((1,0), 1025).
cell((1,1), 6672).
cell((1,2), 0).
cell((0,0), 0).
cell((0,1), 16386).
cell((0,2), 1025).
```

The mined valid trajectories are:

Trajectory 1: path(east,5,move_forward,east,6) path(east,6,turn_left,north,1) path(north,1,move_forward,east,2) path(east,2,move_forward,east,3) path(east,3,move_forward,south,8) path(south,8,move_forward,east,9)

Trajectory 2: path(east,5,move_forward,east,6) path(east,6,turn_right,south,11) path(south,11,move_forward,east,12) path(east,12,move_forward,east,13) path(east,13,move_forward,north,8) path(north,8,move_forward,east,9)

Trajectory 3:
path(west,9,move_forward,west,8) path(west,8,turn_left,north,3) path(north,3,move_forward,west,2) path(west,2,move_-forward,west,1) path(west,1,move_forward,south,6) path(south,6,move_forward,west,5)

Trajectory 4:
path(west,9,move_forward,west,8) path(west,8,turn_right,south,13) path(south,13,move_forward,west,12) path(west,12,move_-forward,west,11) path(west,11,move_forward,north,6) path(north,6,move_forward,west,5)

As you can see, each trajectory is clenched together by **unique matching pairs of outgoing / incoming directions / positions**:

... path(west,11,move_forward,**north,6**) path(**north,6**,move_forward,west,5) ...

6.5 'Transition' Atoms

Next, we needed an atom to comprehensively track each train's position, storing a complete set of movement attributes for subsequent control.

Unique train ID - given in the input, usually an integer.

Global moment of time - the moment when a transition occurs. Strictly speaking, it relates to the moment of time when the train transits between two cells united by a 'path' bracket. However, for convenience of interpretation (the reader will see later, when we introduce the custom visualizer), we may view it as a moment when the train occupies the first of two unified cells. The same moment of time 'ticks' for all trains in the system, whether it is obtained from a global or a local (train-specific) time horizon.

Incoming position - 1d position of the cell that the train enters.

Incoming direction - direction from which the train enters the cell.

Outgoing position - 1d position which the train would enter from the current cell. This direction, together with the outgoing direction, is used to **tie two transitions within one trajectory**.

Outgoing direction - direction from which the train enters the next cell. Together with 'outgoing position', it ties two adjacent transitions within one trajectory.

Action - action that was committed to make the indicated transition. Unlike a perfectly naive generator, **the action is deterministic given the path/trajectory**. All these attributes were combined in a 'transition' atom:

```
transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR)
```

6.6 The Generator

The core architectural pattern of all answer set programs contains just three parts:

1. Describing the problem,
2. Generating solution candidates,
3. Prohibiting solution candidates that do not satisfy certain conditions.

While valid trajectory mining with 'path' atoms focused on narrowing the answer search space as much as possible, the 'transition' atoms described above were generated in stage 2 using a choice rule [5] to produce all possible combinations over the input, constrained by in-rule conditions or cardinality constraints [5]. In the Table 2, you can see our implementation.

Table 2: Choice Rule Implementation

Choice Rule Idea	**ASP Implementation**
For each train and each moment of time, produce only 1 (cardinality constraint) transition over the existing path.	`1 { transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, OUT_DIR): path(IN_DIR, FROM_POS, ACTION, OUT_DIR, TO_POS) } 1 : train(TRAIN), moment(TRAIN, MOMENT).`

6.7 Constraints

Once the generator produced all possible combinations of 'transition' atoms within the train-specific upper time bound, we introduced rules to reject erroneous combinations, specifically those where:

1. The train movements are inconsequential given the chosen trajectory.
2. The train starts movement differently from how it is described in the 'start' input fact, including the starting direction, position and time.
3. The train does not arrive at its destination given in the input facts.
4. Two opposite trains swap positions.

In answer set programming, we could conveniently realize all these rules in the form of constraints.

6.8 Waiting

Waiting instead of moving is the simplest form of collision avoidance in a multi-agent dynamic system. It is also the default method used in the ASP implementation of Flatland via the 'wait' action in the answer set.

Since each successive pair of cells in the extracted trajectories had a single associated action, the most reasonable way to implement waiting was to allow it at every point corresponding to a path ('path' atom):

Normal Rule Idea	**ASP Code**
If there is a path between two cells, there must also be an opportunity to wait on this path.	`path(IN_DIR, POSITION, wait, IN_DIR, POSITION):- path(IN_DIR, POSITION, _, _, _).`

Table 3: The idea for a basic 'wait' creating normal rule and its asp implementation

Testing showed that, for collision avoidance, it suffices to allow each train to wait only immediately before each junction (according to Flatland rules, trains 'disappear' at destination stations, so collisions there are impossible). Waiting is also permitted at the departure station, which can be practical to accommodate belated passengers or in project extensions with connected routes, where a delay of one train may require the next train to wait. So, the final 'wait' action creating rule looked as following:

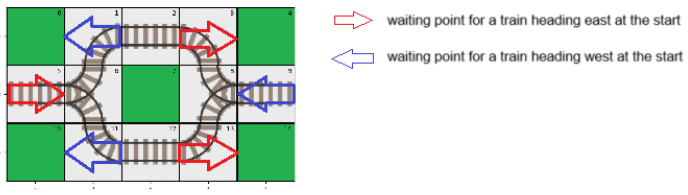
Normal Rule Idea	**ASP Code**
If there is a path between two cells and the second cell leads into a junction, there must also be an opportunity to wait on this path.	`path(IN_DIR, IN_POS, wait, IN_DIR, IN_POS):- path(IN_DIR, IN_POS, _, OUT_DIR, OUT_POS), path(OUT_DIR, OUT_POS, _, _, _), is_junction(OUT_POS).`

Table 4: The idea for 'wait' creating normal rule and its asp implementation

As a result, the simple environment shown above also had the following waiting "path" atoms added to the mined trajectories:

path(east,5,wait,east,5) path(west,1,wait,west,1) path(west,11,wait,west,11) path(west,9,wait,west,9) path(east,3,wait,east path(east,13,wait,east,13)

Which corresponds to the following 'waiting points':



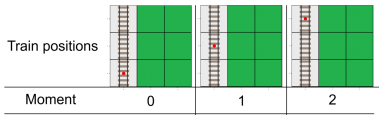
6.9 Custom Test Environments

During development of the basic encoding, we could not immediately rely on the test environments from the project repository[6], as building a robust, well-functioning model is inherently incremental. In our case, the model progressed through the following approximate evolutionary phases:

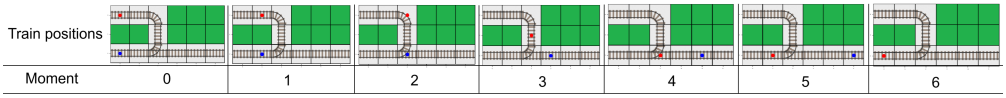
ensuring the train moves ⇒ ensuring it stays on tracks ⇒ ensuring it can choose direction at switches ⇒ ensuring it starts from the indicated departure station and direction ⇒ ensuring it arrives at the indicated

destination ⇒ ensuring arrival before the global time horizon ⇒ ensuring no collisions between trains ⇒ ensuring two opposite trains do not swap positions

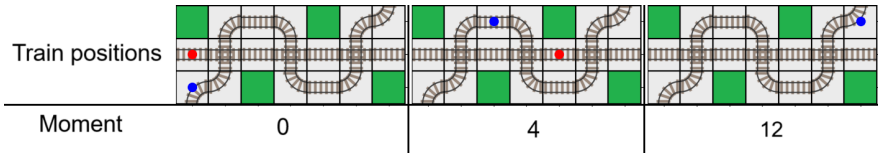
In the frame by frame representations in Figure 2, you can see how our custom environments evolved together with the solving ability of our encoding.



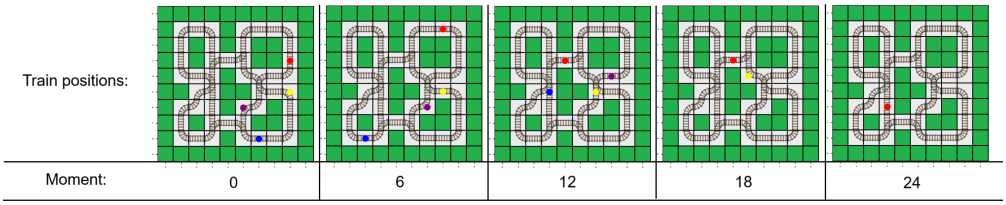
(a) Straight line: a train can move



(b) Simple switch negotiation: two trains can move and choose direction on switch



(c) Multiple diamond crossings: two trains can move and avoid collisions by waiting



(d) Ten by ten with many turns: multiple trains can move and avoid collisions by waiting

Figure 2: Sample motion sequences with trajectory based basic movement model

While developing this basic encoding, we drew all our test environments either by hand or in a graphic editor and then manually described them in terms of 'cell' facts. It was extremely tedious, but we did not find a better solution. Being aware of the existence of Clingonia[7], we failed to use it due to unresolvable dependencies.

Later, when working on the chosen domain area extension, we discovered that our extended solution could not be properly visualized by the default flatland environment[6]. Having explored the Flatland source code, we realized that it would be easier to develop our own visualizer than to change the provided one. Hence, all the visualizations the reader may see in this report or as animations in our git repository are made using our simple custom visualizer.

6.10 Optimality

One of the goals of scheduling, and railway scheduling in particular, is to optimize a function that describes the joined movement of all agents in the system. This optimization task can be viewed as maximization or minimization, for example:

Maximization: *"Find an optimum that maximizes the number of trains arriving on time."*

Minimization: *"Find an optimum that minimizes the sum of individual delays across all trains in the system."*

In the context of train scheduling, minimization appeared more natural, so we chose the following objective:

$$
\begin{aligned}
M^* = \arg\min_{M} \quad & \sum_{t=1}^{n} d_t \\
\text{subject to} \quad & M \text{ represents valid train transitions} \\
& d_t \geq 0 \qquad\qquad \forall t \in \{1, \ldots, n\} \\
& \text{where } \texttt{d_t} \text{ denotes the delay of train } \texttt{t}
\end{aligned}
$$

Which in terms of ASP was expressed as the following imperative[5]:

```
#minimize {1,TRAIN,MOMENT,ACTION : transition(TRAIN, MOMENT, _, _, ACTION, _, _)}.
```

f

6.11 Potential Future Improvements of the Trajectory Mining Approach

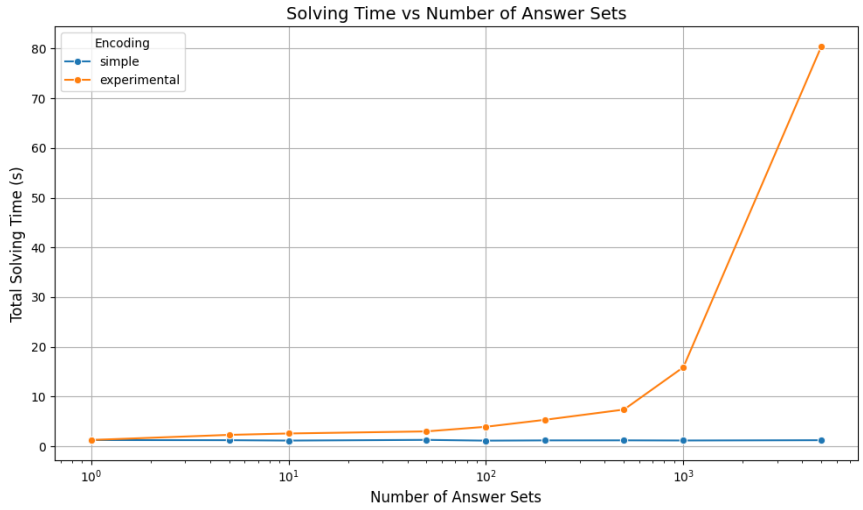
While scheduling trains[1] with asp yields for almost infinite research depth, our work was necessarily limited by one semester. Hence, it is important to indicate potential improvements of our trajectory-based model:

1. Introducing a procedural programming layer (Python) to prune train-irrelevant / inefficient trajectories (for example, using A*) and only generate train positions over the remaining trajectories, radically reducing grounding.
2. Condensing trajectories from the clenched 'path' atom expression to a smaller set of atoms down to just one per trajectory, allowing for a much more precise movement generation.

7 Comparative Testing

To compare our two independent encodings, we tested their solving speeds in identical 40×40 Flatland environments with multiple trains and varied tracks. Only the encoding logic differed - all other variables remained constant for fairness. We began by:

- Measuring solving time using Clingo's built-in timing output
- Repeating tests multiple times to account for variance
- Comparing runtime performance across encodings



The CPU used for these tests was an 13th Gen Intel(R) Core(TM) i5-1335U.

8 Domain Area Extension

Per project rules[1], teams must not only solve collision-free train movement but also implement a previously unaddressed domain problem atop the basic solution. After testing both encodings, we had to select an extension.

8.1 Choice of Extension

We sought an extension that was engaging, feasibly solvable with ASP (e.g., not heavily relying on continuous numbers), and manageable within one semester. Our candidates were:

1. Track inclines (flatland into Switzerland)

One early idea was adding track inclines, modifying the input environment to simulate their effects on trains. This was relevant since Flatland's sponsor, SBB Switzerland operates its trains on the terrain with varying, steep gradients.

After implementing our basic models, we found that adding inclines would only affect train speeds (slowing climbs, accelerating descents). This, in its turn, would require the model to represent dynamically varying speeds, which had not been done by any team before.

2. Prioritization

We also explored train prioritization (e.g., express services), modeled via conflict resolution rules. However, given its limited complexity and prior implementations, we concluded it wouldn't provide sufficient depth for our project.

3. Acceleration

Acceleration—and its counterpart, deceleration—was noted as a particularly challenging unaddressed problem. Noting most prior work focused on macro-level aspects [3] [8] of a network, we deliberately targeted a micro-scale, train-specific challenge for novelty.

8.2 Train Prioritization

Initially, we pursued train prioritization for its practical relevance, implementing a basic ID-based system in 15 minutes. However, given its simplicity and extensive prior work [3] [8], we pivoted to a less explored challenge.

```
1 {action(train(ID),A,T) : action_type(A), A!=wait; action(train(ID),wait,T) :
at(ID,X,Y,T,Dir), offset(Dir,DX,DY), at(ID2,X+DX,Y+DY,T,_), ID > ID2} 1 :- train(ID),
time(T), not goal(ID,T), not reached(ID,T).
```

Figure 3: Implementation of prioritization based on train IDs.

8.3 Acceleration: Overview

Acceleration is fundamental to physical movement: any vehicle accelerates from rest to a certain speed, either uniformly or variably based on propulsion, external conditions or operational constraints (e.g., passenger comfort or cargo safety).

Upon reaching target speed, the vehicle either maintains it indefinitely or decelerates through the inverse process.

8.4 Acceleration: Domain Area Implications

Acceleration and deceleration critically impact rail systems. Excessive forces compromise passenger comfort, potentially causing injuries, fatalities, or lost clients.

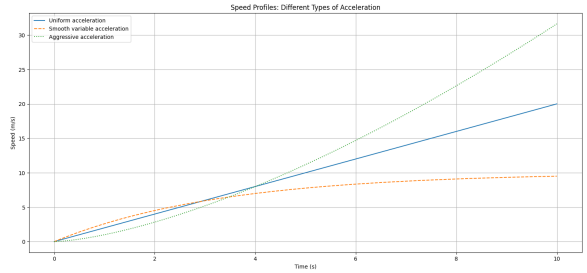


Figure 4: Acceleration profile

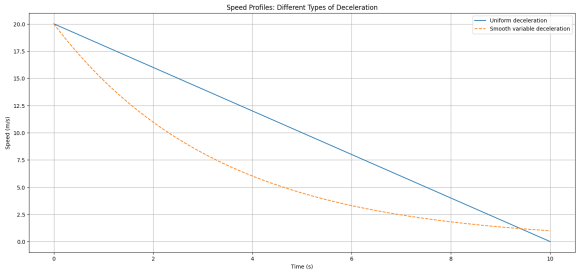


Figure 5: Deceleration profile

During the conceptual phase, we made ourselves aware of railway acceleration standards for passenger systems [9] [10], and freight-specific constraints [11] [10]) governing cargo types, weight, and coupler mechanics.

While modeling these nuances went beyond the project scope, knowing that acceleration and deceleration are subject to strict industry regulations was important to give our simulation realistic boundaries and might be useful for future groups (topic idea: ensuring the most comfortable trip for passengers by planning smooth acceleration and deceleration and avoiding full stops).

Acceleration and deceleration are critical for energy efficiency: bringing a train—especially a heavily loaded freight train—to a complete stop is **significantly more energy-intensive** than decelerating without a full stop: the energy is required not only to decelerate a large mass but also to accelerate it again. From basic physics, the kinetic energy required to bring a mass m moving at speed v to rest is $\frac{1}{2}mv^2$; regaining this speed after a stop requires **re-investing that same energy**. Hence, minimizing full halts contributes directly to improving energy efficiency in real-world rail systems.

8.5 Speed

Acceleration is a *derivative of speed*:

$$a = \frac{dv}{dt}$$

which meant that modeling acceleration first required us to implement speed.

We were aware that some teams[12] [13] had approached speed in the previous years, but decided to not look up their solutions, having a clear design intuition early on and being curious to keep it 'clean' and see where it leads us:

1. Each environment operates on a "global time scale", where time progresses uniformly for all trains. This global time is bounded by either a fixed time horizon or the train's individual schedule (note: if we disallow late arrivals, the distinction doesn't affect the model).

2. Each train has its own "local time scale", which is determined by its assigned input speed.

3. The train's velocity emerges from the difference between its local time scale and the global one. This may sound abstract at first, but it reflects our early intuition — more technical explanation follows in the next sections.

For simplicity, we decided to consider the input speed to be the **cruiser speed** of the respective train and to also bind it theoretically (not reflected in the code, but used for conceptual and visual simplicity) to train types. Table 5 describes this correspondence.

As it was indicated in the project description[1], speed is treated in ASP implementation of Flatland as *fractional speed*. This concept is confusing at first, but we made it more intuitive for us by thinking about it in terms of *frequency* instead, so that speed of every train can be viewed as *'frequency of changing actions'*(we cannot say 'frequency of moving between cells', because *waiting is an action too*).

Train Type	Cruiser Speed
ICE	1
Local	2
Freight	3

Table 5: Train types and their cruiser speeds

While designing our speed modeling with future integration of acceleration in mind, we consulted domain literature and found that, in most operational contexts, trains operate at or very near their maximum permitted speed [9] [10], [14], [11]. That is, the cruising speed effectively coincides with the maximum speed. As a result, our speed simulation assumes that trains are either accelerating up to their cruising speed or decelerating from it, but never exceeding it.

Speaking of ASP implementation, it is easiest to understand our approach by imagining time scales of the whole environment ('real time') and train-specific time scales as different arrays (see Figure 6 for visual intuition).

Real Moments	Action Moments
0	0
1	
2	2
3	
4	4
5	

Figure 6: Example for a local train. Real moments elapse for all trains equally (left array), action moments for the local train skip every second real moment(right array).

The 'real moments' array is derived from the arrival time of a train (or a global time horizon, which makes no difference because what is important is that no moments are skipped and there is enough time for arrival):

```
real_moment(TRAIN, 0..MAX_TIME-1) :- end(TRAIN, (_, _), MAX_TIME).
```

The 'action moments' array is derived from the speed of the train using modulo to derive the amount of skipped real moments:

```
action_moment(TRAIN, MOMENT) :- MOMENT = 0..MAX_TIME-1,
                                end(TRAIN, (_, _), MAX_TIME),
                                train(TRAIN),
                                speed(TRAIN, MOM_PER_POS),
                                R = MOMENT \ MOM_PER_POS,
                                R == 0.
```

To further rely on the intuition of arrays, we can imagine zipping these arrays in Python and applying a boolean operator to mark where there is an entry missing from the train-specific array (the real time array is always full and consistent) but present in the real time array (let's mark it as 'lag', see Figure 7).

Having done this, we can extract 'real transitions' from real moments and 'laggy transitions' from lags: to do this, we simply copied the following transition backwards the number of times that was sufficient to fill the gaps in between the real transitions.

But why do we need these dummy transitions at all? Because the **trains might collide while dwelling** in a position, not only while entering it with a real transition! And so, to preserve functionality of all the scheduling logic implemented and tested in the original trajectory based encoding, we need to simulate dwelling in a particular position due to a low speed.

The resulting model was capable of solving all the provided test environments with trains being assigned different speeds.

Real Moments	Action Moments	Transitions
0	real moment	real transition
1	lag	laggy transition
2	real moment	real transition
3	lag	laggy transition
4	real moment	real transition
5	lag	laggy transition

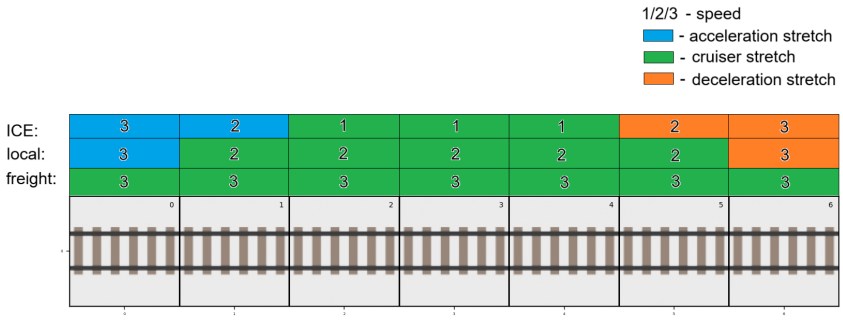
Figure 7: Example for a local train. Presence or absence of an action moment at a certain real moment defines the type of created transition.

8.6 Acceleration: Implementation

Acceleration naturally stemmed from how we implemented speed: if speed is frequency of changing actions, then acceleration / deceleration is *change of the frequency of changing actions*. Since our implementation of speed was based on relative frequencies of 'real moments' and 'all moments', its extension into acceleration followed naturally: to simulate acceleration and deceleration, we just needed to space 'action atoms' unevenly.

Real Moments	Action Moments
0	0
1	
2	
3	3
4	
5	5
6	6
7	7
8	8
9	9
10	10
11	
12	12
13	
14	
15	15

(a) Spacing of action moments for an ICE vs real time scale. The acceleration stretch, the cruiser stretch and the deceleration stretch are easily found



(b) Deterministic acceleration/deceleration bound to specific train types in our simulation.

Figure 8: Acceleration phases and deterministic properties for different train types.

Technically, all we needed to do was to provide a mechanism that would output differently spaced 'action moment', with spacing tied to the train type. The reader will be introduced to the technical details of this part later in the text.

8.7 Acceleration: Levels of Complexity

The task of implementing acceleration, while conceptually clear, still seemed overwhelming at first. To make it more approachable, we decided to break it down in two levels of complexity, eventually adding more:

8.8 Deterministic Acceleration

It is clear that any moving vehicle, to reach point B from point A, must first accelerate from rest to a constant speed and later decelerate to a full stop at point B. In the simplest case—where speed remains constant between these two phases—the lengths of the acceleration and deceleration segments are determined solely by the vehicle's properties.

We took this simplest scenario as the basic implementation of acceleration and deceleration, where these properties of the system were determined solely from the type of the train. Under this naive approach, the speed profiles (see Figure 9) have fixed acceleration and deceleration lengths, while the cruising segment length varies with the distance to the destination.

The initial acceleration for every train was defined by the its departure time given in the input facts and its type. It was implemented simply as a set of train-type specific recursive normal rules, as in this ICE example:

```
action_moment(TRAIN, 0) :- speed(TRAIN, _).
action_moment(TRAIN, 3) :- speed(TRAIN, 1), action_moment(TRAIN, 0).
action_moment(TRAIN, 5) :- speed(TRAIN, 1), action_moment(TRAIN, 3).
```

Creation of the cruiser stretch action moments was then also done recursively, as addition (the added amount depended on the train type, i.e. cruiser speed) to the last moment of the acceleration stretch:

```
action_moment(TRAIN, NEXT_ACTION_MOMENT) :- speed(TRAIN, 1),
                                            action_moment(TRAIN, PREV_ACT_MOMENT),
                                            max_time(TRAIN, MAX_TIME),
                                            PREV_ACT_MOMENT >= 5,
                                            braking_time(TRAIN, BRAKING_TIME),
                                            max_time(TRAIN, MAX_TIME),
                                            NEXT_ACTION_MOMENT = PREV_ACT_MOMENT + 1,
                                            NEXT_ACTION_MOMENT < MAX_TIME + 1
                                            - BRAKING_TIME.
```

While beginning of the acceleration was simply tied to the start of movement, the beginning of the deceleration required an **external signal**. Such a signal could be obtained in two different ways:

- Spatial braking signal: *'start braking when there is a certain distance to the destination / obstacle'*
- Temporal braking signal: *'start braking a certain amount of moments before a destination / obstacle'*

This is close to a real railway system, where a braking signal comes to the train driver when the train approaches a certain point in space or time. The distinction between spatial and temporal triggering aligns with how real-world systems trade off between track-based detection (e.g., balises, signal blocks) and train-internal time-based predictions.

In our initial implementation, we decided to bind 'braking signal' creation to the train-specific time horizon given in the 'end' input fact:

```
start_braking(TRAIN, ACTION_MOMENT) :- max_time(TRAIN, MAX_TIME),
                                       speed(TRAIN, _),
                                       braking_time(TRAIN, BRAKING_TIME),
                                       action_moment(TRAIN, ACTION_MOMENT),
                                       ACTION_MOMENT == MAX_TIME - BRAKING_TIME.
```

Where 'braking time' auxiliary atom was derived from the train's cruiser speed (implicitly, from the train type) and indicated the number of moments necessary for each train type to completely stop:

```
braking_time(TRAIN, 5) :- speed(TRAIN, 1). % ICE
braking_time(TRAIN, 3) :- speed(TRAIN, 2). % Local
braking_time(TRAIN, 3) :- speed(TRAIN, 3). % Freight
```

While these braking times might not be absolutely realistic, they reflect the truth of a fast train taking longer to stop, and are easily refined into a more precise simulation, with the only implication being the computation speed.

Once the unevenly spaced 'action moment' atoms were created by this recursive logic, the rest of the model - 'real transition' and 'laggy transition' atom creation together with constraints - worked smoothly without any changes, ensuring timely arrival and collision avoidance. You can enjoy the animated output of this acceleration model on a 10x10 environment with 3 different trains (ICE, local, freight) - the varying speed is most noticeable if you follow trains one by one.

This approach allowed us to prove that our intuition of modeling acceleration in asp as 'change of the frequency of change' was true, together with our 'time-scale mismatch' core logic. Let's put a bottom line to this simple stage:

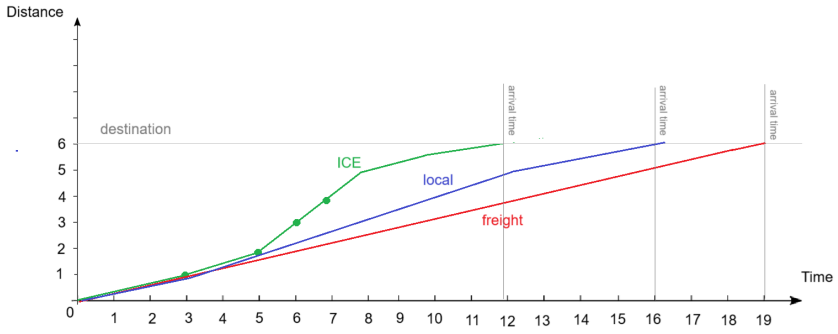


Figure 9: Speed profiles of different train types (ICE high-speed, local, and freight trains) over time on a short 6 cell track. Acceleration (rising slopes) and deceleration phases (falling slopes) are clearly distinguishable. The difference in slope in the middle sections corresponds to the difference in cruiser speeds of different train types.

Advantages: makes the entire simulation more realistic, since initial acceleration and final deceleration are inevitable in any real transportation system (while intermediate accelerations/decelerations are not bound to happen when trains are scheduled carefully).

Disadvantages:

1. only models acceleration and deceleration at a certain time span from the departure and before the arrival, does not provide the opportunity to use these properties to avoid obstacles;

2. neglects the fact that waiting must be preceded by a gradual deceleration and followed by a gradual acceleration - if a train has to wait on the path, it will just stop and then return to its cruiser speed abruptly, without any intermediate speed values.

8.9 Stochastic Acceleration

Solving a problem with answer set programming implies producing all stable models that satisfy certain constraints[5]. When applied to acceleration and deceleration as mechanisms for resolving train conflicts, this entails producing all possible combinations of acceleration phases, deceleration phases, and cruising segments for each train. In the Figure 10, you can observe a small subset of possible combinations of accelerations, decelerations and cruiser speed stretches on a straight track of just 10 cells.

The optimal solution is then selected by minimizing the total amount of waiting time across all trains in the environment:

$$\text{solution}^* = \arg\min \sum_{t \in T} \text{wait}_t$$

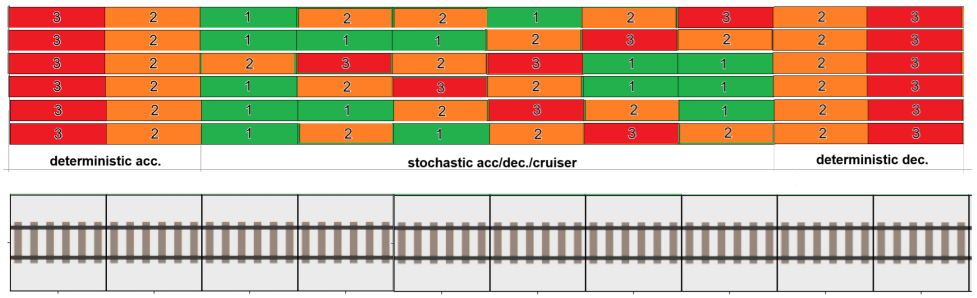


Figure 10: Subset of ICE stochastic acceleration values on a track of 10 cells. Notice that initial acceleration and final deceleration stretches are deterministic.

The intuition illustrated in Figure 10 suggests that the number of possible combinations of acceleration patterns is extremely large. Nevertheless, let us now formally define the computational complexity of applying fully stochastic acceleration:

Let X be a one-dimensional (straight track) environment of size 1×10 (see Figure 10).

Assume there is one train of type `ICE`, which allows three discrete velocity states (we omit the full stop state to simplify the analysis): $\frac{1}{1}, \frac{1}{2}, \frac{1}{3}$.

The deterministic segments of the trajectory (highlighted in red and labeled "deterministic acc." and "deterministic dec.") together occupy 4 cells. These sections follow a fixed pattern and contribute a computational complexity that is linear in the number of deterministic steps:

$$\mathcal{O}_{\text{det}} = \text{len}(\text{deterministic segment}) = 4$$

The remaining 6 cells form the stochastic segment, where the train can accelerate, decelerate, or cruise, provided that transitions respect a step-wise constraint. For example, a transition between speed $\frac{1}{1}$ and $\frac{1}{3}$ must pass through $\frac{1}{2}$, and vice versa. This introduces a combinatorial component to the complexity due to the number of valid speed sequences over these 6 steps.

To simplify notation, we represent speeds by the denominator of their fractional form:

$$S = \{1, 2, 3\}$$

Transitions are only allowed between adjacent speeds (step-wise), i.e., a valid transition must satisfy:

$$|s_i - s_{i+1}| \leq 1$$

A Markov transition matrix modeling allowed switches is shown in Figure 11a.

Let $L = 6$ be the number of cells in the stochastic segment. We now wish to count the number of all valid sequences $s_1, s_2, \ldots, s_L \in S^L$ such that:

$$\forall i \in \{1, \ldots, L-1\}, \quad |s_i - s_{i+1}| \leq 1$$

This problem is equivalent to counting all walks of length L in a graph defined by the Markov matrix (Figure 11b). A valid walk may start at any node in S.

Then, the number of valid sequences of length L (including the starting point) can be computed by summing all entries in A^{L-1}, as each entry (i, j) of A^{L-1} gives the number of walks from state i to state j in $L - 1$ steps:

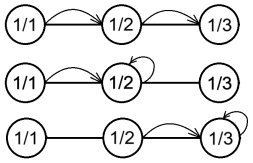
$$N_{\text{valid}}(L) = \sum_{i,j} \left(A^{L-1}\right)_{i,j}$$

For $L = 6$, this results in:

$$N_{\text{valid}}(6) = \sum_{i,j} \left(A^{5}\right)_{i,j}$$

	1	2	3
1	1	1	0
2	1	1	1
3	0	1	1

(a) A Markov matrix for ICE speed switches given the switch must be stepwise.



(b) Examples of Markov chains modeling valid speed switches of an ICE. A switch can start from any node. Maintaining a certain speed is possible (diagonal of the Markov matrix).

Figure 11: Markov-based modeling of valid ICE speed switches.

The problem is that $N_{\text{valid}}(L)$ grows exponentially with the length L of the stochastic segment, since at each step the number of valid transitions is governed by the powers of the transition matrix A. As a result, for longer tracks or multiple trains, the total number of valid sequences quickly becomes astronomically large:

$$N_{\text{valid}}(L) = \sum_{i,j} \left(A^{L-1}\right)_{i,j} = \mathcal{O}(\lambda_{\max}^{L-1})$$

where $\lambda_{\max}$ is the largest eigenvalue of the matrix A (essentially, *how many options we're branching into*).

Although implementing fully stochastic acceleration is feasible within narrowly constrained settings (e.g., for individual train paths of length up to 10), its practical applicability to realistically sized environments appears highly limited.

8.10 Constrained Stochastic Acceleration

While the rigid deterministic acceleration only gave increased realism to the model but did not provide additional control for collision avoidance, and the fully stochastic model had inescapable computational costs due to combinatorial explosion, the middle ground was found in form of **constrained stochastic acceleration** - an approach where every train could change its speed freely, but only once in between two junctions.

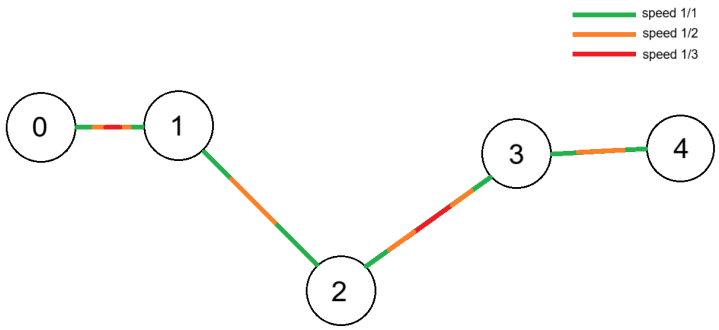


Figure 12: Schematic representation of constrained stochastic acceleration. Nodes represent junctions in the environment. Edges represent paths between junctions. Color-coding denotes possible fluctuations of speed in between each pair of junctions, which for an ICE train implies 3 degrees of freedom.

By 'change of speed' we mean not a single degree (e.g., from 1/3 to 1/2), but the full depth of acceleration/deceleration available for the train according to its type. So, the rule states: *"A train may switch to any speed within the range available to it and then return to its cruiser speed, but only once within a section in between every two junctions"*.

The benefit of this approach lies in its ability to use speed fluctuations to avoid collisions while keeping combinatorial complexity at bay.

Implementation-wise, we had most of the support structure in place by the moment we designed this solution: for instance, 'is junction' atoms were implemented when working on the basic encoding - they ensured that the train could only wait before junction and not anywhere else.

Due to the inherent complexity and many interdependencies of the model, we adopted a very iterative development. Initially, we implemented a simple prototype that handled a one-dimensional straight track with a few junctions—these junctions served merely as triggers without leading anywhere (see Figure 13).

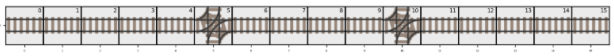


Figure 13: 1d environment with junctions as triggers

The core idea was straightforward: trains were allowed to change their speeds only at junctions. This aligns well with practical railroad operations, where control actions typically occur just before junctions. Consequently, in our encoding, external control over the train's "gear" was modeled such that each junction along the route could trigger a speed change. To represent this, we defined a choice rule with two possible options:

```
speed_signal(drop_speed; keep_speed).
```

We also needed to parse junctions via simple graph property rules and then assign different speed control signals to different parsed stretches:

```
1{speed_control((FIRST, SECOND), SIGNAL): speed_signal(SIGNAL)}1 :-
varying_speed_stretch(FIRST, SECOND).
```

Stretches of the environment that included the departure point and the arrival point were made speed-deterministic to avoid redundant switching, which was expressed via a normal rule:

```
speed_control((FIRST, SECOND), keep_speed) :- fixed_speed_stretch(FIRST, SECOND).
```

Local, stretch-specific action moments were created from speed control atoms and then mapped to the global train-specific scale, so that the spacing of action moments could only differ in intervals corresponding to parsed junctions.

The rest of the logic worked smoothly with this implementation, leading to a solution where train could switch speed arbitrarily at the junctions.

Once this basic prototype worked, it seemed that generalizing it to work with two-dimensional environments was a matter of a few lines of code - it happened to be a very wrong expectation.

The first natural step was to expand the edge parsing logic. We achieved this relatively quickly by describing graph properties more exhaustively in auxiliary atoms:

```
edge((Y,X1), (Y,X2)) :- edge_candidate((Y,X1), (Y,X2)),
                        not gap_between((Y,X1), (Y,X2)),
                        not junction_between((Y,X1), (Y,X2)),
                        X1 != X2.
```

Combined with the previously described control signal logic, it allowed us to obtain a very clear foundation for train routing in the form of answer sets containing different combinations of parsed edges and speed control signals on them (see Figure 14).

The reason why we included train IDs (first attribute) into speed control atoms was the domain-inspired heuristic of not allowing the train to change speed right after departure and before arrival.

At this point, it seemed that we were close to a solution where the number of action atoms per edge for a particular train would be dependent on that stretch's local density of action moments. However, we hit the wall here: the amount of produced action moments was dependent on the total length of the whole environment and not on the length of the train's relevant trajectory. We had, consequently, an abundance of redundant action moments for every train that made meaningful production of real transition atoms impossible:

```
Answer: 1
speed_control(2,(12,14),keep_speed) speed_control(2,(14,17),keep_speed) speed_control(2,(0,2),keep_speed) speed_control(2,(2,14),drop_speed)
Answer: 2
speed_control(2,(12,14),keep_speed) speed_control(2,(14,17),keep_speed) speed_control(2,(0,2),keep_speed) speed_control(2,(2,14),keep_speed)
Answer: 3
speed_control(2,(12,14),keep_speed) speed_control(2,(14,17),keep_speed) speed_control(2,(0,2),drop_speed) speed_control(2,(2,14),drop_speed)
Answer: 4
speed_control(2,(12,14),keep_speed) speed_control(2,(14,17),keep_speed) speed_control(2,(0,2),drop_speed) speed_control(2,(2,14),keep_speed)
SATISFIABLE
```

Figure 14: Speed control atoms obtained from a simple environment.

```
1 { real_transition(TRAIN, MOMENT, FROM_POS, IN_DIR, ACTION, TO_POS, OUT_DIR)
: path(IN_DIR, FROM_POS, ACTION,
OUT_DIR, TO_POS) } 1 :- train(TRAIN), action_moment(TRAIN, MOMENT).
```

After tens of hours of failed attempts to prune action moments production to the train relevant trajectories (i.e. through running path-collecting 'pathfinders' first or integrating cell-action-moment binding into the choice rule above), we decided to temporarily step aside and approach meaningful speed variation differently.

8.11 Temporal Stochastic Acceleration

Contrary to our previous belief on the fully stochastic acceleration (see Section 8.9), it appeared that binding acceleration and deceleration of trains to the global or local time horizon is a far easier approach than binding it to 'geography' of the network: instead of parsing graph structure and assigning different control signals with a choice rule (see Section 8.10), we only needed to produce different sets of action moments (see Figure 10) whose count would be bound to a time horizon. Each set was supposed to contain a number of action moments such that space between them could be uneven, but only within the range allowed by the train type. To achieve this, we needed to use local fluctuations of speed.

Together, this led to the following code structure which combined a choice rule for the local speed value and a recursive creation of the next action moment using the chosen speed:

```
spacing(TRAIN, MOM_PER_ACTION..3) :- speed(TRAIN, MOM_PER_ACTION).

action_moment(TRAIN, DEPARTURE_MOMENT) :- start(TRAIN, _, DEPARTURE_MOMENT, _).

1{local_speed(TRAIN, ACTION_MOMENT, LOCAL_SPEED): spacing(TRAIN, LOCAL_SPEED)}1 :-
action_moment(TRAIN, ACTION_MOMENT).

action_moment(TRAIN, NEXT_ACTION_MOMENT) :- action_moment(TRAIN, PREV_ACTION_MOMENT),
                                            local_speed(TRAIN, PREV_ACTION_MOMENT, SPACING),
                                            NEXT_ACTION_MOMENT = PREV_ACTION_MOMENT + SPACING,
                                            global(GLOBAL_TIME_HORIZON),
                                            NEXT_ACTION_MOMENT < GLOBAL_TIME_HORIZON.
```

Once the sets of differently spaced action moments were created (for better intuition, see Figure 10 again), we could proceed to the movement logic, borrowing it fully from our speed model and ending up with a simpler code structure than even the one we had built for deterministic acceleration.

The constraints we used starting from the basic movement encoding and all the way into deterministic acceleration also held.

We added, however, one element to this part - a constraint effectively setting up the minimum number of times that each speed available for the given train type must appear in the solution:

```
min_moms_each_speed(MIN).
```

Where the variable served to set up the minimum amount of moments a train was to supposed to hold each of the speed value available to it according to its type.

```
train_speed_count(TRAIN, SPEED, COUNT) :-
    train(TRAIN),
    speed_value(TRAIN, SPEED),
    COUNT = #count { MOMENT : local_speed(TRAIN, MOMENT, SPEED) }.

:- train_speed_count(_, _, COUNT),
   min_moms_each_speed(MIN),
   COUNT <= MIN.
```

Finally, since the practical motivation for us was to minimize waiting across all trains in the environment and rely on speed fluctuations for routing instead, we expressed this explicitly as a minimization statement:

```
#minimize {1,TRAIN,TIME : real_transition(TRAIN, TIME,_,_,wait,_,_)}.
```

As a result, we obtained an encoding that could solve (in the example the ICE (blue) accelerates to let the local (red) pass a complex junction) complex multi-train environments using train type specific changes of speed.

Potential improvements:

1. It appears easy to combine this logic with our deterministic acceleration - so that the solving power of temporal stochastic acceleration would benefit from extended realism of initial acceleration / final slow down.
2. An additional control mechanism can be implemented to avoid 'jerks' - overly short (down to 1 moment) fluctuations of speed that would be undesirable in a real network.

References

[1] Murphy and Schaub. "Railway Scheduling Using Answer Set Programming". Project course, Potsdam University. 2025.

[2] Dirk Abels et al. "Train Scheduling with Hybrid ASP". In: (2023). Affiliations: Potassco Solutions (Germany), SBB (Switzerland), University of Potsdam (Germany).

[3] Julius Gotsch and Isabel Seidl. *Prioritization*. University of Potsdam, Am Neuen Palais 10; 14469 Potsdam; Germany. Email: goetsch@uni-potsdam.de, seidl1@uni-potsdam.de. 2023. URL: https://....

[4] Jonathan Drathjer, Arne Frenzel, and Leander von Gottherg. *Railway Scheduling Report*. Tech. rep. Email: drathjer@uni-potsdam.de, frenzel5@uni-potsdam.de, gottherg@uni-potsdam.de. Potsdam, Germany: Institut für Informatik, Universität Potsdam, 2023. URL: https://....

[5] Martin Gebser et al. *Answer Set Solving in Practice*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan and Claypool Publishers, 2012. ISBN: 978-1-60845-991-7. DOI: 10.2200/S00457ED1V01Y201211AIM019.

[6] University of Potsdam KRR Group. *Flatland: Answer Set Programming (ASP) System*. https://github.com/krr-up/flatland. Version Latest commit: <COMMIT-HASH>. Accessed: 2023-10-25. 2023.

[7] Max Vogel. *Clingonia*. https://github.com/M4X-VOGEL/Clingonia. 2023.

[8] Nicole Mc Duffie and Theodore Sither. *Prioritization in Answer Set Programming for Railway Scheduling*. Tech. rep. Technical Report. Am Neuen Palais 10, 14469 Potsdam, Germany: Universität Potsdam, 2025. URL: https://github.com/tsither/amtrak.

[9] *Railway applications – Ride comfort for passengers – Measurement and evaluation*. German. Standard. English version: EN 12299:2009. Berlin, Germany: Deutsches Institut für Normung, Mar. 2009. URL: https://www.beuth.de/en/standard/din-en-12299/114003331 (visited on 11/15/2023).

[10] *SBB CFF FFS 482.01: Regulation for Railway Operations and Safety*. french/german/italian. Swiss railway operations standard. Bern, Switzerland: Swiss Federal Railways (SBB CFF FFS), 2023.

[11] *VDV 750: Fahrwegkonflikte im Schienenverkehr*. Köln, Germany: Verband Deutscher Verkehrsunternehmen (VDV), 2023.

[12] Jonathan Drathjer, Arne Frenzel, and Leander von Gottberg. *Railway Scheduling with Variable Train Speeds in Answer Set Programming*. Tech. rep. Potsdam, Germany: University of Potsdam, 2025. URL: https://github.com/....

[13] Alexej Lesser and Arne Jacoby. *Railway Scheduling with Speed Variations and Malfunctions in ASP*. Tech. rep. Potsdam, Germany: University of Potsdam, 2025. URL: https://github.com/arugula/AStar_Flatland.

[14] Swiss Federal Railways (SBB). *RIL 400.2101: [Exact Title in Original Language]*. German. Technical Regulation. Internal regulation code: 400.2101. Bern, Switzerland: SBB Infrastructure, [Year]. URL: %5BOfficial%20URL%20if%20publicly%20available%5D (visited on 11/20/2023).